Name	Engineering
REACTOR	Diagnose/remedy reactor accidents
DELTA	Diagnose/remedy GE locomotives
STEAMER	Instruct operation—steam powerplant

Table 1.7 Engineering Expert Systems

Name	Geology
DIPMETER	Interpret dipmeter logs
LITHO	Interpret oil well log data
MUD	Diagnose/remedy drilling problems
PROSPECTOR	Interpret geologic data for minerals

Table 1.8 Geology Expert Systems

Name	Computer Systems
PTRANS	Give prognosis for managing DEC computers
BDS	Diagnose bad parts in switching net
XCON	Configure DEC computer systems
XSEL	Configure DEC computer sales orders
XSITE	Configure customer site for DEC computers
YES/MVS	Monitor/control IBM MVS operating system
TIMM	Diagnose DEC computers

Table 1.9 Computer Expert Systems

Appropriate Domains for Expert Systems

Before starting to build an expert system, it is essential to decide if an expert system is the appropriate paradigm. For example, one concern is whether an expert system should be used instead of an alternative paradigm, such as conventional programming. The appropriate domain for an expert system depends on a number of factors:

- *Can the problem be effectively solved by conventional programming?* If the answer is yes, then an expert system is not the best choice. For example, consider the problem of diagnosing some equipment. If all the symptoms for all malfunctions are known in advance, then a simple table lookup or decision tree of the fault is adequate. Expert systems are best suited for situations in which there is no efficient algorithmic solution. Such cases are called **ill-structured problems** and reasoning may offer the only hope of a good solution.

As an example of an ill-structured problem, consider the case of a person who is thinking about travel and visits a travel agent. While most people have a destination and plans, there are exceptions. Table 1.10 lists some characteristics of an ill-structured problem as indicated by the person's responses to the travel agent's questions.

Although this is probebly an extreme case, it does illustrate the basic concept of an ill-structured problem. As you can see, an ill-structured problem would not lend itself well to an algorithmic solution because there are so many possibilities. In this case, a default option should be exercised when all else fails. For example,

the travel agent could say, "Ah ha! I have the perfect trip for you: a round-the-world cruise. Please fill out the following credit card application and everything will be taken care of."

Travel Agent's Questions	Responses
Can I help you?	I'm not sure.
Where do you want to go?	Somewhere.
Any particular destination?	Here and there.
How much can you afford?	I don't know.
Can you get some money?	I don't know.
When do you want to go?	Sooner or later.

Table 1.10 An Example of an Ill-structured Problem

In dealing with ill-structured problems, there is a danger in that the expert system design may accidentally mirror an algorithmic solution. That is, the development of the expert system may unknowingly discover an algorithmic solution. An indication that this has happened is if a solution that requires a rigid **control structure** is found. That is, the rules are forced to execute in a certain sequence by the knowledge engineer explicitly setting the priorities of many rules. Forcing a rigid control structure on the expert system cancels a major advantage of expert system technology, which is dealing with unexpected input that does not follow a predetermined pattern (Parrello 88). That is, expert systems react opportunistically to their input, whatever it is. Conventional programs generally expect input to follow a certain sequence. An expert system with a lot of control often indicates a disguised algorithm and may be a good candidate for recoding as a conventional program.

- *Is the domain well-bounded?* It is important to have well-defined limits on what the expert system is expected to know and what its capabilities should be. For example, suppose you wanted to create an expert system to diagnose headaches. Certainly the medical knowledge of a physician would be put in the knowledge base. However, for a deeper understanding of heedaches, you might also put in knowledge about neurochemistry, then its parent area of biochemistry, then chemistry, molecular biophysics, and so forth perhaps on down to subnuclear physics. Other domains such as biofeedback, psychology, psychiatry, physiology, exercise, yoga, and stress management may also contain pertinent knowledge about headaches. The point: when do you stop adding domains? The more domains, the more complex the expert system becomes.

In particular, the task of coordinating all the expertise becomes a major task. In the real world we know from experience how difficult it is to have coordinated teams of experts working on problems, especially when they come up with conflicting recommendations. If we knew how to program well the coordination of expertise, then we could try programming an expert system to have the knowledge of multiple experts. Attempts have been made to coordinate multiple expert systems in the HEARSAY II and HEARSAY III systems. However, such coordination is a complex task that should be viewed more as research rather than as producing a deliverable expert system product.

- *Is there a need and a desire for an expert system?* Although it's great experience to build an expert system, it's rather pointless if no one is willing to use it. If there are many human experts already, it's difficult to justify an expert system based on the reason of scarce human expertise. Also, if the experts or users don't want the system, it will not be accepted even if there is a need for it.

Management especially must be willing to support the system. This is even more critical for expert systems than for conventional programs because expert systems is a new technology. This means that there are few experienced people and more uncertainty about what can be accomplished. However, the area of expert systems deserves more support because it attempts to solve the problems that cannot be solved by conventional programming. The risks are greater, but so are the rewards.

- *Is there at least one human expert who is willing to cooperate?* There must be an expert who is willing, and preferably enthusiastic, about the project. Not all experts are willing to have their knowledge examined for faults and then put into a computer. Even if there are multiple experts willing to cooperate in the development, it might be wise to limit the number of experts involved. Different experts may have different ways of solving a problem, such as requesting different diagnostic tests. Sometimes they may even reach different conclusions. Trying to code multiple methods of problem solving in one knowledge base may create internal conflicts and incompatibilities.
- *Can the expert explain the knowledge so that it is understandable by the knowledge engineer?* Even if the expert is willing to cooperate, there may be difficulty in expressing the knowledge in explicit terms. As a simple example of this difficulty, can you explain in words how you move a finger? Although you could say it's done by contracting a muscle in the finger, the next question is, how do you contract a finger muscle? The other difficulty in communication between expert and knowledge engineer is that the knowledge engineer doesn't know the technical terms of the expert. This problem is particularly acute with medical terminology. It may take a year or longer for the knowledge engineer to even understand what the expert is talking about, let alone translate that knowledge into explicit computer code.
- *Is the problem-solving knowledge mainly heuristic and uncertain?* Expert systems *are* appropriate when the expert's knowledge is largely heuristic and uncertain. That is, the knowledge may be based on experience, called **experiential knowledge**, and the expert may have to try various approaches in case one doesn't work. In other words, the expert's knowledge may be a trial-and-error approach, rather than one based on logic and algorithms. However, the expert can still solve the problem faster than someone who is not an expert. This is a good application for expert systems. If the problem can be solved simply by logic and algorithms, it is best handled by a conventional program.

1.8 LANGUAGES, SHELLS, AND TOOLS

A fundamental decision in defining a problem is deciding how best to model it. Sometimes experience is available to aid in choosing the best paradigm. For example, experience suggests that a payroll is best done using conventional procedural programming. Experience also suggests that it is preferable to use a commercial package, if available, rather than writing one from scratch. A general guide to selecting a paradigm is to consider the most traditional one first—conventional programming. The reason for doing this is because of the vast amount of experience we have with conventional programming and the wide variety of commercial packages available. If a problem cannot be effectively done by conventional programming, then turn to nonconventional paradigms such as AI.

Although expert systems is a branch of AI, there are specialized languages for expert systems that are quite different from the commonly used AI languages such as LISP and PROLOG. Although many others have been developed, such as IPL-II, SAIL, CONNIVER, KRL, and Smalltalk, few are widely used except for research (Scown 85).

An expert system language is a higher-order language than languages like LISP or C because it is easier to do certain things, but there is also a smaller range of problems that can be addressed. That is, the specialized nature of expert system languages makes them suitable for writing expert systems but not for general purpose programming. In many situations, it is even necessary to exit temporarily from an expert system language to perform a function in a procedural language.

The primary functional difference between expert system languages and procedural languages is the focus of representation. Procedural languages focus on providing flexible and robust techniques to represent data. For example, data structures such as arrays, records, linked lists, stacks, queues, and trees are easily created and manipulated. Modern languages such as Modula-2 and Ada are designed to aid in **data abstraction** by providing structures for encapsulation such as modules and packages. This provides a level of abstraction that is then implemented by methods such as operators and control statements to yield a program. The data and methods to manipulate it are tightly interwoven. In contrast, expert system languages focus on providing flexible and robust ways to represent knowledge. The expert system paradigm allows two levels of abstraction: data abstraction and **knowledge abstraction**. Expert system languages specifically separate the data from the methods of manipulating the data. An example of this separation is that of facts (data abstraction) and rules (**knowledge abstraction**) in a rule-based expert system language.

This difference in focus also leads to a difference in program design methodology. Because of the tight interweaving of data and knowledge in procedural languages, programmers must carefully describe the sequence of execution. However, the explicit separation of data from knowledge in expert system languages requires considerably less rigid control of the execution sequence. Typically, an entirely separate piece of code, the inference engine, is used to apply the knowledge to the data. This separation of knowledge and data allows a higher degree of parallelism and modularity.

In choosing a language, a basic question should be whether the problem is knowledge or intelligence intensive. Expert systems rely on a great deal of specialized knowledge or expertise to solve a problem, whereas AI emphasizes a problem-solving approach. Expert systems often rely on a pattern matching within a restricted knowledge domain to guide their execution whereas AI generally concentrates on searching paradigms in less restricted domains.

The customary way of defining the need for an expert system program is to decide if you want to program the expertise of a human expert. If such an expert exists and will cooperate, then an expert system approach may be successful.

The road to selecting an expert system language is paved with confusion. A few years ago the choice of an expert system language was fairly straightforward. There were only about a half dozen languages available, and they were generally free or cost a nominal amount payable to the universities where they were developed.

However, with the explosive commercial growth in the expert systems field since the 1970s, the selection of a language is no longer so simple. Today there are dozens of languages available, ranging in price up to $75,000. Although it is still possible to obtain some of the older languages such as OPS5 free or at a nominal cost from the universities where they were developed, there is a price to pay in terms of efficiency, lack of modern features, and support.

Besides the confusing choice of the many languages available today, the terminology used to describe the languages is confusing. Some vendors refer to their products as "tools," while others refer to "shells" and still others talk about "integrated environments." For clarity in this book, the terms will be defined as follows:

- **Language**: A translator of commands written in a specific syntax. An expert system language will also provide an inference engine to execute the statements of the language. Depending on the implementation, the inference engine may provide forward chaining, backward chaining, or both. Under this language definition, LISP is not an expert system language but PROLOG is. However, it is possible to write an expert system language using LISP and to write AI in PROLOG. For that matter, you can even write an expert system or AI language in assembly language. Questions of development time, convenience, maintainability, efficiency, and speed determine which language software is written in.
- **Tool**: A language plus associated utility programs to facilitate the development, debugging, and delivery of application programs. Utility programs may include text and graphics editors, debuggers, file management, and even code generators. Cross-assemblers may also be provided to port the developed code to different hardware. For example, an expert system may be developed on a DEC VAX and then cross-assembled to run on a Motorola 68000. Some tools may even allow the use of different paradigms such as forward and backward chaining in one application.

In some cases a tool may be integrated with all its utility programs in one environment to present a common interface to the user. This approach minimizes the need for the user to leave the environment to perform a task. For example, a simple tool may not provide facilities for file management and so a user would have to exit the tool to give operating system commands. An integrated environment allows easy exchange of data between utility programs in the environment. Some tools do not even require the user to write any code. Instead, the tool allows a user to enter knowledge by examples from tables or spreadsheets and generates the appropriate code itself.

- **Shell**: A special-purpose tool designed for certain types of applications in which the user must supply only the knowledge base. The classic example of this is the EMYCIN (empty MYCIN) shell. This shell was made by removing the medical knowledge base of the MYCIN expert system.

MYCIN was designed as a backward chaining system to diagnose diseases. By simply removing the medical knowledge, EMYCIN was created and could be used as a shell to contain knowledge about other kinds of consultative systems that use backward chaining. The EMYCIN shell demonstrated the reusability of the essential MYCIN software such as the inference engine and the user interface. This was a very important step in the development of modern expert system technology because it meant that an expert system would not have to be built from scratch for each new application.

There are many ways of characterizing expert systems such as representation of knowledge, forward or backward chaining, support of uncertainty, hypothetical reasoning, explanation facilities, and so forth. Unless a person has built a number of expert systems, it is difficult to appreciate all of these features, especially those found in the more expensive tools. The best way to learn expert systems technology is to develop a number of systems with an easy-to-learn language and then invest in a more sophisticated tool if you need its features.

1.9 ELEMENTS OF AN EXPERT SYSTEM

The elements of a typical expert system are shown in Figure 1.6. In a rule-based system the knowledge base contains the domain knowledge needed to solve problems coded in the form of rules. While rules are a popular paradigm for representing knowledge, other types of expert systems use different representations, as discussed in Chapter 2.

An expert system consists of the following components:

- **user interface**—the mechanism by which the user and the expert system communicate.
- **explanation facility**—explains the reasoning of the system to a user.
- **working memory**—a **global database** of facts used by the rules.
- **inference engine**—makes inferences by deciding which rules are satisfied by facts or objects, prioritizes the satisfied rules, and executes the rule with the highest priority.
- **agenda**—a prioritized list of rules created by the inference engine, whose patterns are satisfied by facts or objects in working memory.
- **knowledge acquisition facility**—an automatic way for the user to enter knowledge in the system instead of having the knowledge engineer explicitly code the knowledge.

The knowledge acquisition facility is an an optional feature on many systems. In some expert system tools like KEE and First Class the tool can learn by rule induction through examples and can automatically generate rules. However, the examples are generally from tabular or spreadsheet-type data better suited to

decision trees. General rules constructed by a knowledge engineer can be much more complex than the simple rules from rule induction.

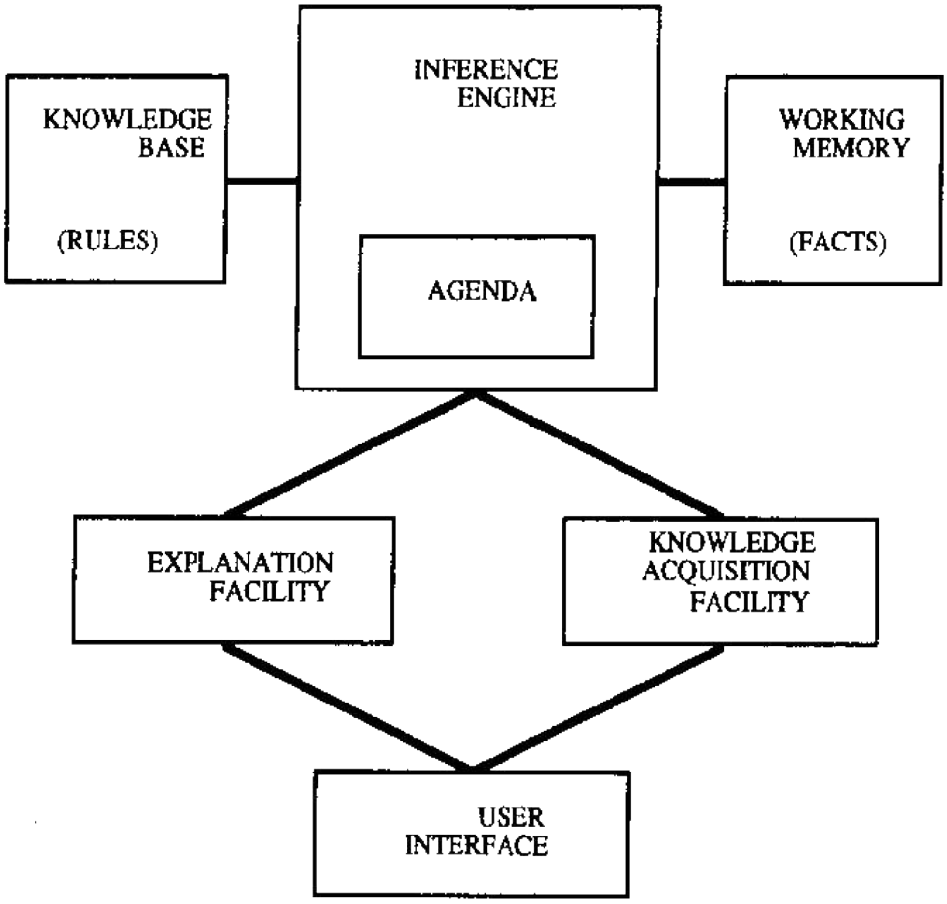


Figure 1.6 Structure of a Rule-based Expert System

Depending on the implementation of the system, the user interface may be a simple text-oriented display or a sophisticated, high-resolution, bit-mapped display. This high-resolution display is commonly used to simulate a control panel with dials and displays.

The knowledge base is also called the **production memory** in a rule-based expert system. As a simple example, consider the problem of deciding to cross a street. The productions for the two rules are as follows, where the arrows mean that the system will perform the actions on the right of the arrow if the conditions on the left are true.

```
the light is red → stop
the light is green → go
```

The production rules can be expressed in an equivalent pseudocode IF...THEN format as:

```
Rule: Red_light
IF
       the light is red
THEN
       stop

Rule: Green_light
IF
       the light is green
THEN
       go
```

Each rule is identified by a name, followed by the IF part of the rule. The section between the IF and THEN parts of the rule is called by various names including the **antecedent**, **conditional part**, **pattern part**, or **left-hand-side** (**LHS**). The individual condition

```
the light is green
```

is called a **conditional element** or a **pattern**.

Some examples of rules from real systems are:

```
MYCIN system for diagnosis of meningitis and
bacteremia (bacterial infections)
IF
    The site of the culture is blood, and
    The identity of the organism is not known with
       certainty, and
    The stain of the organism is gramneg, and
    The morphology of the organism is rod, and
    The patient has been seriously burned
THEN
    There is weakly suggestive evidence (.4) that
       the identity of the organism is pseudomonas

XCON/R1 for configuring DEC VAX computer systems
IF
    The current context is assigning devices to
       Unibus modules and
    There is an unassigned dual-port disk drive and
    The type of controller it requires is known and
    There are two such controllers, neither of
       which has any devices assigned to it, and
    The number of devices that these controllers
       can support is known
```

```
THEN
    Assign the disk drive to each of the
       controllers, and
    Note that the two controllers have been
       associated and each supports one drive
```

In a rule-based system, the inference engine determines which rule antecedents, if any, are satisfied by the facts. Two general methods of inferencing are commonly used as the problem-solving strategies of expert systems: **forward chaining** and **backward chaining**. Other methods used for more specific needs may include means-ends analysis, problem reduction, backtracking, plan-generate-test, hierarchical planning and the least commitment principle, and constraint handling.

Forward chaining is reasoning from facts to the conclusions resulting from those facts. For example, if you see that it is raining before leaving home (the fact) then you should take an umbrella (the conclusion).

Backward chaining involves reasoning in reverse from a hypothesis, a potential conclusion to be proved, to the facts that support the hypothesis. For example, if you have not looked outside and someone enters with wet shoes and an umbrella, your hypothesis is that it is raining. In order to support this hypothesis you could ask the person if it was, in fact, raining. If the response is yes, then the hypothesis is proved true and becomes a fact. As mentioned before, a hypothesis can be viewed as a fact whose truth is in doubt and needs to be established. The hypothesis can then be interpreted as a goal to be proved.

Depending on the design, an inference engine will do either forward or backward chaining. For example, OPS5 and CLIPS are designed for forward chaining whereas EMYCIN performs backward chaining. Some types of inference engines, such as ART and KEE, offer both. The choice of inference engine depends on the type of problem. Diagnostic problems are better solved with backward chaining whereas prognosis, monitoring, and control are better accomplished by forward chaining.

The working memory may contain facts regarding the current status of the traffic light such as "the light is green" or "the light is red." Either or both of these facts may be in working memory at the same time. If the traffic light is working normally, only one fact will he in memory. However, it is possible that both facts may he in working memory if there is a malfunction in the light. Notice the difference between the knowledge base and working memory. Facts do not interact with one another. The fact "the light is green" has no effect on the fact "the light is red." Insteed, our knowledge of traffic lights says that if both facts are simultaneously present, then there is a malfunction in the light.

If there is a fact "the light is green" in working memory, the inference engine will notice that this fact satisfies the conditional part of the green light rule and will put this rule on the agenda. If a rule has multiple patterns, then all of its patterns must be simultaneously satisfied for it to be placed on the agenda. Some patterns may even be satisfied by specifying the absence of certain facts in working memory.

A rule whose patterns are all satisfied is said to be **activated** or **instantiated**. Multiple activated rules may he on the agenda at the same time, in which case the inference engine must select one rule for **firing**. The term *firing* comes from neurophysiology, the study of the nervous system. An individual nerve cell or neuron emits an electrical signal when stimulated. No further amount of stimulation can cause the neuron to fire again for a short time period. This

phenomenon is called *refraction*. Rule-based expert systems are built using refraction in order to prevent trivial loops. That is, if the green light rule kept firing on the same fact over and over again, the expert system would never accomplish any useful work.

Various methods have been invented to provide refraction. In one type of expert system language called OPS5, each fact is given a unique identifier called a **timetag** when it is entered in working memory. After a rule has fired on a fact, the inference engine will not fire on that fact again because its time stamp has been used.

Following the THEN part of a rule is a list of **actions** to be executed when the rule fires. This part of the rule is known as the **consequent** or **right-hand side (RHS)**. When the red light rule fires, its action, stop, is executed. Likewise, when the green light rule fires, its action is go. Specific actions usually include the addition or removal of facts from working memory or printing results. The format of these actions depends on the syntax of the expert system language. For example, in OPS5, ART, and CLIPS the action to add a new fact called "stop" to working memory would be (assert stop). Because of their LISP ancestry, these languages were designed to require parentheses around patterns and actions.

The inference engine operates in **cycles**. Various names have been given to describe the cycle such as **recognize-act cycle**, **select-execute cycle**, **situation-response cycle**, and **situation-action cycle**. By any name for a cycle, the inference engine will repeatedly execute a group of tasks until certain criteria cause execution to cease. The tasks of a cycle for OPS5, a typical expert system shell, are shown in the following pseudocode as **conflict resolution**, **act**, **match**, and **check for halt**.

WHILE not done

> **Conflict Resolution**: If there are activations, then select the one with the highest priority else done.
>
> **Act**: Sequentially perform the actions on the RHS of the selected activation. Those that change working memory have an immediate effect in this cycle. Remove the activation that has just fired from the agenda.
>
> **Match**: Update the agenda by checking whether the LHSs of any rules are satisfied. If so, activate them. Remove activations if the LHSs of their rules are no longer satisfied.
>
> **Check for Halt**: If a halt action is performed or a break command given, then done.
>
> END-WHILE
>
> **Accept a new user command**

Multiple rules may be activated and put on the agenda during one cycle. Also, activations will be left on the agenda from previous cycles unless they are deactivated because their LHS is no longer satisfied. Thus the number of activations on the agenda will vary as execution proceeds. Depending on the program, an activation may always be on the agenda but never selected for firing. Likewise, some rules may never become activated. In these cases, the purposes of these rules should be re-examined because either the rules are unnecessary or their patterns were not correctly designed.

The inference engine executes the actions of the highest priority activation on the agenda, then the next highest priority activation, and so on until no activations

are left. Various priority schemes have been designed into expert system shells. Generally all shells let the knowledge engineer define the priority of rules.

Agenda conflicts occur when different activations have the same priority and the inference engine must decide on one rule to fire. Different shells have different ways of dealing with this problem. In the original Newell and Simon paradigm, those rules entered in the system first had the highest default priority (Newell 72, p. 33). In OPS5, rules with more complex patterns have higher priority. In ART and CLIPS, rules have the same default priority unless assigned different ones by the knowledge engineer.

At this time, control is returned to the **top-level** command interpreter for the user to give further instructions to the expert system shell. The top-level is the default mode in which the user communicates with the expert system and is indicated by the task "Accept a new user command." It is the top-level that accepts the new command.

The top-level is the user interface to the shell while an expert system application is under development. More sophisticated user interfaces are usually designed for the expert system to facilitate its operation. For example, the expert system may have a user interface for control of a manufacturing plant that shows a block diagram of the plant with a high-resolution, bit-mapped color display. Warnings and status messages may appear in flashing colors with simulated dials and gauges. In fact, more effort may go into the design and implementation of the user interface than in the expert system knowledge base, especially in a prototype. Depending on the capabilities of the expert system shell, the user interface may be implemented by rules or in another language called by the expert system.

An explanation facility will allow the user to ask how the system came to a certain conclusion and why certain information is needed. The question of how the system came to a certain conclusion is easy to answer in a rule-based system because a history of the activated rules and contents of working memory can be maintained in a stack. Sophisticated explanation facilities may allow the user to ask "What If"-type questions to explore alternate reasoning paths through hypothetical reasoning.

1.10 PRODUCTION SYSTEMS

One of the most popular types of expert systems today is the rule-based system. Rules are popular for a number of reasons.

- *Modular nature.* This makes it easy to encapsulate knowledge and expand the expert system by incremental development.
- *Explanation facilities.* It is easy to build explanation facilities with rules because the antecedents of a rule specify exactly what is necessary to activate the rule. By keeping track of which rules have fired, an explanation facility can present the chain of reasoning that led to a certain conclusion.
- *Similarity to the human cognitive process.* Based on the work of Newell and Simon, rules appear to be a natural way of modeling how humans solve problems. The simple IF...THEN representation of rules makes it easy to explain to experts the structure of the knowledge you are trying to elicit from them. Other advantages of rules are described in Hayes-Roth (85).

Rules are a type of production whose origins go back to the 1940s. Because of the importance of rule-based systems, it is worthwhile to examine the development of the rule concept. This will give you a better idea why rule-based systems are so useful for expert systems.

Post Production Systems

Production systems were first used in symbolic logic by Post, who originated the name (Post 43). He proved the important and amazing result that any system of mathematics or logic could be written as a certain type of production rule system. This result established the great capability of production rules for representing major classes of knowledge rather than being limited to a few types. Under the term **rewrite rules**, they are also used in linguistics as a way of defining the grammar of a language. Computer languages are commonly defined using the Backus-Naur Form (BNF) of production rules.

Post's basic idea was that any mathematic or logic system is simply a set of rules specifying how to change one string of symbols into another set of symbols. That is, given an input string, the antecedent, a production rule could produce a new string, the consequent. This idea is also valid with programs and expert systems in which the initial string of symbols is the input data and the output string is some transformation of the input.

As a very simple case, suppose the input string is "patient has fever," the output string might then be "take an aspirin." Note that there is no meaning attached to these strings. That is, the manipulations of the strings is based on syntax and not on any semantics or understanding of what *fever, aspirin,* and *patient* represent. A human knows what these strings in terms of the real world mean but a Post production system is just a way of transforming one string into another. A production rule for this example could be

```
Antecedent → Consequent
person has fever → take aspirin
```

where the arrow indicates the transformation of one string into another. We can interpret this rule in terms of the more familiar IF...THEN notation as

```
IF person has fever THEN take aspirin
```

The production rules can also have multiple antecedents. For example,

```
person has fever AND
        fever is greater than 102 → see doctor
```

Note that the special connective AND is not part of the string. The AND indicates that the rule has multiple antecedents.

A Post production system consists of a group of production rules, such as the following (the numbers in parentheses are for purposes of this discussion):

```
(1)   car won't start → check battery
(2)   car won't start → check gas
(3)   check battery AND battery bad → replace
      battery
(4)   check gas AND no gas → fill gas tank
```

If there is a string "car won't start," the rules (1) and (2) may be used to generate the strings "check battery" and "check gas." However, there is no control mechanism that applies both these rules to the string. Only one rule may be applied, or both, or none. If there is another string "battery bad" and a string "check battery," then rule (3) may be applied to generate the string "replace battery."

There is no special significance to the order in which rules are written. The rules of our example could also have been written in the following order, and the system would be the same system:

```
(4)   check gas AND no gas → fill gas tank
(2)   car won't start → check gas
(1)   car won't start → check battery
(3)   check battery AND battery bad → replace
      battery
```

Although Post production rules were useful in laying part of the foundation of expert systems, they are not adequate for writing practical programs. The basic limitation of Post production rules in programming is lack of a **control strategy** to guide the application of the rules. A Post system permits the rules to be applied on the strings in any manner because there is no specification given on how the rules should be applied.

As an analogy, suppose you go to the library to find a certain book on expert systems. At the library, you start randomly looking at books on the shelves for the one you want. If the library is fairly large, it could take a very long time to find the book you need. Even if you find the section of books on expert systems, your next random choice could take you to an entirely different section, such as on French cooking. The situation becomes even worse if you need material from the first book to help you determine the second book you need to find. A random search for the second book will also take a long time.

Markov Algorithms

The next advance in applying production rules was made by Markov, who specified a control structure for production systems (Markov 54). A **Markov algorithm** is an ordered group of productions that are applied in order of priority to an input string. If the highest priority rule is not applicable then the next one is applied, and so forth. The Markov algorithm terminates if either (1) the last production is not applicable to a string or (2) a production that ends with a period is applied.

Markov algorithms can also be applied to substrings of a string, starting from the left. For example, the production system consisting of the single rule

```
AB → HIJ
```

when applied to the input string GABKAB produces the new string GHIJKAB. Since the production now applies to the new string, the final result is GHIJKHIJ.

The special character ∧ represents the **null string** of no characters. For example, the production

```
A → ∧
```

deletes all occurrences of the character A in a string.